

ESSENTIAL SQL QUERIES FOR EVERY DEVELOPER

1. Write an SQL query to fetch "FIRST_NAME" from Worker table using the alias name as <WORKER_NAME>.

```
SELECT FIRST_NAME AS WORKER_NAME  
FROM WORKER;
```

2. Write an SQL query to fetch "FIRST_NAME" from Worker table in upper case.

```
SELECT upper(FIRST_NAME)  
FROM WORKER;
```

3. Write an SQL query to fetch unique values of DEPARTMENT from Worker table.

```
SELECT distinct(DEPARTMENT)  
FROM WORKER;
```

4. Write an SQL query to print the first three characters of FIRST_NAME from Worker table.

```
SELECT left(FIRST_NAME,3) FROM  
WORKER;  
SELECT substring(FIRST_NAME,1,3) FROM  
WORKER;
```

5. Write an SQL query to find the position of the alphabet („O") in the first name column „GEORGE" from Worker table.

```
SELECT instr(FIRST_NAME, "O")  
FROM WORKER  
WHERE FIRST_NAME="GEORGE";
```

6. Write an SQL query to print the FIRST_NAME from Worker table after removing white spaces from the right side.

```
SELECT rtrim(FIRST_NAME)  
FROM WORKER;
```

7. Write an SQL query to print the DEPARTMENT from Worker table after removing white spaces from the left Side.

```
SELECT ltrim(DEPARTMENT)  
FROM WORKER;
```

8. Write an SQL query that fetches the unique values of DEPARTMENT from Worker table and prints its length.

```
SELECT distinct(DEPARTMENT),  
length(DEPARTMENT)  
FROM WORKER;
```

9. Write an SQL query to print the FIRST_NAME from Worker table after replacing „a“ with „A“.

```
SELECT replace(FIRST_NAME, "I", "J")  
  
FROM WORKER  
  
WHERE FIRST_NAME="IAN";
```

10. Write an SQL query to print the FIRST_NAME and LAST_NAME from Worker table into a single column COMPLETE_NAME.

```
SELECT  
concat(FIRST_NAME,",",LAST_NAME) AS  
  
COMPLETE_NAME  
  
FROM WORKER;
```

11. Write an SQL query to print the FIRST_NAME from Worker table after removing white spaces.

```
SELECT trim(FIRST_NAME)  
  
FROM WORKER;
```

12. Write an SQL query to print all Worker details from the Worker table order by FIRST_NAME Ascending.

```
SELECT * FROM WORKER  
  
ORDER BY FIRST_NAME ASC;
```

13. Write an SQL query to print all Worker details from the Worker table order by.
-- FIRST_NAME Ascending and DEPARTMENT Descending--.

```
SELECT * FROM WORKER  
  
ORDER BY FIRST_NAME ASC, DEPARTMENT  
DESC;  
  
SELECT * FROM WORKER  
  
ORDER BY FIRST_NAME, DEPARTMENT DESC;
```

14. Write an SQL query to print details for Workers with the first name as “ETHAN” and “IAN” from Worker table.

```
SELECT * FROM WORKER  
  
WHERE FIRST_NAME IN ("ETHAN", "IAN");
```

15. Write an SQL query to print details of workers excluding first names, “ETHAN” and “IAN” from Worker table.

```
SELECT * FROM WORKER  
  
WHERE FIRST_NAME NOT IN ("ETHAN","IAN");
```

16. Write an SQL query to print details of Workers with DEPARTMENT name as "FINANCE*".

```
SELECT * FROM WORKER
WHERE DEPARTMENT LIKE "FINANCE%";
```

17. Write an SQL query to print details of the Workers whose FIRST_NAME contains „a“.

```
SELECT * FROM WORKER
WHERE FIRST_NAME LIKE "A%";
```

18. Write an SQL query to print details of the Workers whose FIRST_NAME ends with „N“.

```
SELECT * FROM WORKER
WHERE FIRST_NAME LIKE "%N";
```

19. Write an SQL query to print details of the Workers whose FIRST_NAME ends with „A“ and contains six alphabets.

```
SELECT * FROM WORKER
WHERE FIRST_NAME LIKE "%____A";
```

20. Write an SQL query to print details of the Workers whose SALARY lies between 60000 and 100000.

```
SELECT * FROM WORKER
WHERE SALARY between 60000 AND 100000;
```

21. Write an SQL query to print details of the Workers who have joined in Feb 2014.

```
SELECT * FROM WORKER
WHERE year(JOINING_DATE) = 2014
AND month(JOINING_DATE) = 02;
```

22. Write an SQL query to fetch the count of employees working in the department „Admin“.

```
SELECT
DEPARTMENT, COUNT(DEPARTMENT)
FROM WORKER
WHERE DEPARTMENT="HR";
```

23. Write an SQL query to fetch worker full names with salaries >= 50000 and <= 100000.

```
SELECT concat(FIRST_NAME, " ", LAST_NAME)
FROM WORKER
WHERE SALARY between 50000 AND 100000;
```

24. Write an SQL query to fetch the no. of workers for each department in the descending order.

```
SELECT DEPARTMENT, count(WORKER_ID)
AS NO_OF_WORKERS

FROM WORKER

group by DEPARTMENT

ORDER BY NO_OF_WORKERS DESC;
```

25. Write an SQL query to show only odd rows from a table.

```
SELECT * FROM WORKER

WHERE mod(WORKER_ID,2)<>0;
```

26. Write an SQL query to show only even rows from a table.

```
SELECT * FROM WORKER

WHERE mod(WORKER_ID,2)=0;
```

27. Write an SQL query to clone a new table from another table.

```
CREATE TABLE WORKER_CLONE LIKE
WORKER;

INSERT INTO WORKER_CLONE
SELECT * FROM WORKER;

SELECT * FROM WORKER_CLONE;
```

28. Write an SQL query to fetch intersecting records of two tables.

```
SELECT WORKER.* FROM WORKER

inner join WORKER_CLONE

USING(WORKER_ID);
```

29. Write an SQL query to show records from one table that another table does not have.

```
SELECT WORKER.* FROM WORKER
LEFT JOIN WORKER_CLONE
USING(WORKER_ID)
WHERE WORKER_CLONE.WORKER_ID IS
NULL;
```

30. Write an SQL query to show the current date and time.

```
SELECT curdate();

SELECT now();
```

31. Write an SQL query to show the top n (say 5) records of a table order by descending salary.

```
SELECT * FROM WORKER
ORDER BY SALARY DESC
LIMIT 5;
```

32. Write an SQL query to determine the nth (say n=5) highest salary from a table.

```
SELECT * FROM WORKER  
ORDER BY SALARY DESC  
LIMIT 4,1;
```

33. Write an SQL query to determine the 5th highest salary without using LIMIT keyword.

```
SELECT SALARY FROM WORKER W1  
WHERE 4=(  
SELECT COUNT(DISTINCT (W2.SALARY))  
FROM WORKER W2  
WHERE W2.SALARY>=W1.SALARY  
);
```

34. Write an SQL query to fetch the list of employees with the same salary.

```
SELECT W1.* FROM WORKER W1,  
WORKER W2  
WHERE W1.SALARY=W2.SALARY  
AND W2.SALARY!=W1.SALARY;
```

35. Write an SQL query to show the second highest salary from a table using sub-query.

```
SELECT max(SALARY) FROM WORKER  
WHERE SALARY NOT IN (SELECT  
max(SALARY) FROM WORKER);
```

36. Write an SQL query to show one row twice in results from a table.

```
SELECT * FROM WORKER  
UNION ALL  
SELECT * FROM WORKER  
ORDER BY WORKER_ID;
```

37. Write an SQL query to list worker_id who does not get bonus.

```
SELECT WORKER_ID FROM  
WORKER WHERE WORKER_ID NOT  
IN (SELECT  
WORKER_REF_ID FROM BONUS);
```

38. Write an SQL query to fetch the first 50% records from a table.

```
SELECT * FROM WORKER
WHERE WORKER_ID <= ( SELECT
COUNT(WORKER_ID/2) FROM WORKER);
```

39. Write an SQL query to fetch the departments that have less than 4 people in it.

```
SELECT count(DEPARTMENT) AS
DEPCOUNT, DEPARTMENT FROM WORKER
GROUP BY DEPARTMENT
HAVING DEPCOUNT <4;
```

40. Write an SQL query to show all departments along with the number of people in there.

```
SELECT DEPARTMENT,
count(DEPARTMENT) AS DEPCOUNT
FROM WORKER
GROUP BY DEPARTMENT;
```

41. Write an SQL query to show the last record from a table.

```
SELECT * FROM WORKER
WHERE WORKER_ID = (SELECT
MAX(WORKER_ID) FROM WORKER);
```

42. Write an SQL query to fetch the first row of a table.

```
SELECT * FROM WORKER
WHERE WORKER_ID = (SELECT
MIN(WORKER_ID) FROM WORKER);
```

43. Write an SQL query to fetch the last five records from a table.

```
SELECT * FROM WORKER
ORDER BY WORKER_ID DESC
LIMIT 5;
```

44. Write an SQL query to print details of the Workers who are also Managers. 45. Write an SQL query to fetch number (more than 1) of same titles in the ORG of different types.

```
SELECT W.* FROM WORKER AS W
INNER JOIN TITLE AS T
ON W.WORKER_ID = T.WORKER_REF_ID
WHERE T.WORKER_TITLE = 'MANAGER'
```

```
SELECT WORKER_TITLE, COUNT(*) AS COUNT
FROM TITLE
GROUP BY WORKER_TITLE
HAVING COUNT > 1;
```


